EXPERIMENT 11: Word Sense Disambiguation (Lesk), Dependency Parsing, and CYK Parsing

AIM:

To implement three NLP tasks: (1) Word Sense Disambiguation using the Lesk Algorithm, (2) Dependency Parsing using a shift-reduce arc-standard parser, and (3) Syntactic parsing using the CYK (Cocke-Younger-Kasami) algorithm with a CNF grammar.

PROCEDURE:

EXERCISE 1 (WSD – Lesk Algorithm):

1. Define a dictionary with senses for ambiguous words (bank, bat).
2. Implement lesk(word, sentence): tokenize sentence, find sense with maximum gloss overlap.
3. Test with two sentences using the word "bank".

EXERCISE 2 (Dependency Parsing):

1. Define DependencyParser class with stack, buffer, arcs.
2. Implement SHIFT (push buffer to stack), LEFT-ARC, RIGHT-ARC operations.
3. parse() iterates: if stack < 2, SHIFT; else if buffer non-empty, SHIFT; else RIGHT-ARC.
4. Test with "I love NLP", print dependency arcs.

EXERCISE 3 (CYK Parsing):

1. Define grammar in CNF and lexicon.
2. Implement cyk_parser(sentence): fill n×n table bottom-up.
3. Test with "the cat chased the dog", verify if sentence derives S.

CODE:

```
# EXERCISE 1 - Lesk Algorithm (WSD)
import numpy as np

dictionary = {
    "bank": ["financial institution that stores money", "land alongside a river"],
    "bat": ["flying mammal", "equipment used in baseball"]
}

def tokenize(text): return text.lower().split()

def lesk(word, sentence):
    context = tokenize(sentence)
    best_sense = None
    max_overlap = 0
    if word not in dictionary: return None
```

```
    for sense in dictionary[word]:
        gloss = tokenize(sense)
        overlap = len(set(context) & set(gloss))
        if overlap > max_overlap:
            max_overlap = overlap
            best_sense = sense
    return best_sense

sentence1 = "He deposited money in the bank"
sentence2 = "The river bank was full of fish"
print("Sentence:", sentence1)
print("Sense:", lesk("bank", sentence1))
print("\nSentence:", sentence2)
print("Sense:", lesk("bank", sentence2))

# EXERCISE 2 - Dependency Parser
SHIFT = 0; LEFT_ARC = 1; RIGHT_ARC = 2

class DependencyParser:
    def __init__(self):
        self.stack = []; self.buffer = []; self.arcs = []
    def initialize(self, sentence):
        self.stack = ["ROOT"]; self.buffer = sentence.split(); self.arcs = []
    def parse(self):
        while len(self.buffer) > 0 or len(self.stack) > 1:
            if len(self.stack) < 2: self.shift()
            else:
                if len(self.buffer) > 0: self.shift()
                else: self.right_arc()
        return self.arcs
    def shift(self): self.stack.append(self.buffer.pop(0))
    def left_arc(self):
        head = self.stack[-1]; dep = self.stack[-2]
        self.arcs.append((head, dep)); del self.stack[-2]
    def right_arc(self):
        head = self.stack[-2]; dep = self.stack[-1]
        self.arcs.append((head, dep)); self.stack.pop()

parser = DependencyParser()
parser.initialize("I love NLP")
arcs = parser.parse()
print("\nDependency Arcs:")
for arc in arcs: print(arc)

# EXERCISE 3 - CYK Parser
grammar = {("Det","N"):"NP",("V","NP"):"VP",("NP","VP"):"S"}
lexicon = {"the":["Det"],"cat":["N"],"dog":["N"],"chased":["V"]}

def cyk_parser(sentence):
    words = sentence.lower().split()
    n = len(words)
    table = [[[] for _ in range(n)] for _ in range(n)]
    for i, word in enumerate(words):
        if word in lexicon: table[i][i] = lexicon[word]
    for l in range(2, n+1):
        for i in range(n-l+1):
            j = i + l - 1
            for k in range(i, j):
                for A in table[i][k]:
                    for B in table[k+1][j]:
                        if (A,B) in grammar:
                            table[i][j].append(grammar[(A,B)])
    return table
```

```
sentence = "the cat chased the dog"
table = cyk_parser(sentence)
print("\nCYK Table:")
for row in table: print(row)
print("\nSentence derives S:", "S" in table[0][-1])
```

OUTPUT:

```
Sentence: He deposited money in the bank
Sense: financial institution that stores money

Sentence: The river bank was full of fish
Sense: land alongside a river

Dependency Arcs:
('love', 'NLP')
('I', 'love')
('ROOT', 'I')

CYK Table:
[['Det'], ['NP'], [], [], ['S']]
[[], ['N'], [], [], []]
[[], [], ['V'], [], ['VP']]
[[], [], [], ['Det'], ['NP']]
[[], [], [], [], ['N']]

Sentence derives S: True
```

RESULT:

All three NLP tasks were successfully implemented. The Lesk Algorithm correctly disambiguated 'bank' based on context overlap. The Dependency Parser produced correct arc relations for 'I love NLP'. The CYK parser confirmed that 'the cat chased the dog' derives the start symbol S under the given CNF grammar.